



OC Pizza

HandlePizza

Dossier d'exploitation

Version 2

Auteur

Antoine de Chaunac
Analyste programmeur

DCAC
DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com
S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxx – SIREN 999 999 999 – Code APE :
6202A



TABLE DES MATIERES

1 - Versions	4
2 - Introduction	5
2.1 - Objet du document	5
2.2 - Références.....	5
3 - Pré-requis d'exploitation	6
3.1 - Compte, accès et habilitations (AWS/GitHub/IAM)	6
3.2 - Environnements (DEV/TEST-UAT/PROD).....	6
3.3 - Services externes requis (bancaire, fournisseur).....	6
3.4 - Outils d'exploitation (CloudWatch, dashboards, runbook)	7
4 - Procédure de déploiement.....	8
4.1 - Déploiement backend (ECS Fargate).....	8
4.1.1 - Artefacts	8
4.1.2 - Variables d'environnement.....	8
4.1.3 - Configuration applicative	9
4.1.3.1 - Fichier application.yml.....	10
4.1.3.2 - Fichier application-<env>.yml	10
4.1.3.3 - Fichier logback-spring.xml.....	10
4.1.4 - Déploiement	10
4.1.5 - Vérifications post-déploiement	11
4.1.6 - Guide pour le déploiement backend	11
4.1.6.1 - Informations opérationnelles à renseigner lors de la mise en œuvre.....	20
4.2 - Déploiement frontend (S3 + CloudFront + WAF).....	22
4.2.1 - Artefacts	22
4.2.2 - Publication des assets.....	22
4.2.3 - Invalidation cache CloudFront.....	22
4.2.4 - Vérification post-déploiement.....	23
4.2.5 - Guide pour le déploiement frontend.....	23
4.2.5.1 - Informations opérationnelles à renseigner lors de la mise en œuvre.....	31
4.3 - Migrations base de données (Flyway).....	32
4.3.1 - Pré-requis.....	32
4.3.2 - Exécution.....	32
4.3.3 - Controles.....	32
4.3.4 - Rollback / reprise	33
5 - Procédure de démarrage / arrêt.....	34
5.1 - Couche application (ECS).....	34
5.2 - Couche données (RDS, Redis, RabbitMQ).....	35



5.3 - Vérification de santé.....	35
6 - Procédure de mise à jour.....	37
6.1 - Mise à jour applicative.....	37
6.2 - Mise à jour base de données.....	37
6.3 - Mise à jour configuration / secrets.....	38
6.4 - Contrôles post-mise à jour.....	38
6.5 - Guide pour la mise à jour de la solution.....	39
6.5.1 - Informations opérationnelles à renseigner lors de la mise en œuvre	46
7 - Supervision et monitoring.....	48
7.1 - Logs centralisés (CloudWatch).....	48
7.2 - Métriques et alertes	48
7.3 - Supervision des integrations externes	49
7.4 - Gestion d'incident (runbook)	49
8 - Procédure de sauvegarde et restauration	51
8.1 - Politique de sauvegarde.....	51
8.2 - Procédure de restauration.....	51
8.3 - Tests périodiques de restauration	52
8.4 - Guide pour la sauvegarde et la restauration	52
8.4.1 - Informations opérationnelles à renseigner lors de la mise en œuvre	59
9 - Sécurité d'exploitation.....	61
9.1 - Gestion des secrets	61
9.2 - Gestion des accès IAM.....	61
9.3 - Chiffrement et conformité	62
10 - Glossaire	63



1 - VERSIONS

Auteur	Date	Description	Version
ADC	19/05/2026	Création du document	0.1
ADC	28/05/2026	Version 1 du document	1
ADC	09/06/2026	Version 2 du document	2



2 - INTRODUCTION

2.1 - Objet du document

Le présent document constitue le dossier d'exploitation de l'application **HandlePizza** de la société **OC Pizza**.

Il décrit les conditions nécessaires à l'exploitation opérationnelle de la solution, les pré-requis techniques, les procédures de déploiement et de mise à jour, ainsi que les mécanismes de supervision, de sauvegarde et de reprise.

Ce document a pour objectif de :

- Formaliser les règles d'exploitation des environnements DEV, TEST/UAT et PROD ;
- Sécuriser les opérations techniques (déploiement, rollback, maintenance) ;
- Garantir la continuité de service et la fiabilité des traitements métier ;
- Fournir un référentiel commun aux équipes techniques, exploitation et support.

2.2 - Références

Pour de plus amples informations, se référer :

1. **DCT - Dossier_conception_technique** : dossier de conception technique de l'application
2. **DCF - Dossier_conception_fonctionnelle** : dossier de conception fonctionnelle de l'application



3 - PRÉ-REQUIS D'EXPLOITATION

3.1 - Compte, accès et habilitations (AWS/GitHub/IAM)

L'exploitation de **HandlePizza** nécessite les accès suivants :

- Compte AWS du projet avec séparation des droits par rôle (développement, exploitation, administration) ;
- Accès **GitHub** au repository et à la **pipeline GitHub Actions** ;
- **Habilitations IAM** appliquant le principe du moindre privilège ;
- Accès restreint aux secrets techniques via **AWS Secrets Manager** ;
- Traçabilité des accès et actions sensibles (audit).

Les accès de production sont limités aux profils autorisés et soumis à revue périodique.

3.2 - Environnements (DEV/TEST-UAT/PROD)

Trois environnements sont maintenus :

DEV : développement et validation technique rapide ;

TEST/UAT : recette technique et validation métier ;

PROD : exploitation en service réel.

Chaque environnement reprend la même architecture logique (exposition web, backend, données, supervision), avec un dimensionnement et des politiques de sécurité adaptés.

3.3 - Services externes requis (bancaire, fournisseur)

Les services externes nécessaires au fonctionnement sont :

- Système bancaire : autorisation/refus des transactions de paiement ;
- Plateforme fournisseur : échanges de réapprovisionnement et retours de réception.

Pré-requis :

DCAC
DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com
S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



- Endpoints accessibles depuis la couche applicative ;
- Certificats et mécanismes de sécurité valides ;
- Paramètres de timeout/retry/circuit breaker configurés ;
- Supervision des appels sortants activée.

3.4 - Outils d'exploitation (CloudWatch, dashboards, runbook)

L'exploitation s'appuie sur les outils suivants :

- **CloudWatch** : centralisation des logs, métriques et alertes ;
- **dashboards** : visualisation des indicateurs clés (latence, erreurs, charge, disponibilité) ;
- **runbook incident** : procédures de diagnostic, remédiation, escalade et communication.

Ces outils sont requis pour garantir une supervision proactive et une gestion maîtrisée des incidents.



4 - PROCÉDURE DE DÉPLOIEMENT

Cette section décrit les étapes opérationnelles de déploiement de **HandlePizza** sur les environnements cibles.

Elle couvre le déploiement backend et frontend, la gestion des configurations, les migrations de base de données et les contrôles post-déploiement, afin de garantir des mises en production fiables et traçables.

4.1 - Déploiement backend (ECS Fargate)

Le backend de **HandlePizza** est déployé sous forme de **services conteneurisés Spring Boot (Java 17)** exécutés sur **ECS Fargate**.

L'exposition des API est assurée via **ALB en HTTPS**, tandis que la configuration sensible est gérée hors code via **AWS Secrets Manager**.

4.1.1 - Artefacts

Le backend **HandlePizza** est livré sous forme d'image conteneurisée (**Spring Boot Java 17**), construite et versionnée par la **pipeline CI/CD**.

Contrairement à un modèle **batch ZIP**, le déploiement backend repose sur :

- Une image applicative versionnée (tag de release),
- Une définition de **service ECS Fargate**,
- Une configuration d'environnement externalisée,
- Des secrets techniques gérés hors code.

Artefacts principaux :

- Image backend **handlepizza-backend:<version>** ;
- Définition de **tâche ECS** (task definition) ;
- Paramètres d'environnement ;
- **Scripts SQL Flyway** versionnés (si migration DB associée).

4.1.2 - Variables d'environnement

DCAC	100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com
DCAC.COM	S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



Les variables d'environnement backend sont injectées au runtime par environnement (**DEV/TEST-UAT/PROD**) :

Nom	Obligatoire	Description
SPRING_PROFILES_ACTIVE	Oui	Profil Spring actif (dev, test, prod)
SERVER_PORT	Oui	Port applicatif du conteneur
DB_HOST	Oui	Endpoint base de données MySQL
DB_PORT	Oui	Port DB (généralement 3306)
DB_NAME	Oui	Nom du schéma applicatif
DB_USERNAME_SECRET_ARN	Oui	Référence secret login DB
DB_PASSWORD_SECRET_ARN	Oui	Référence secret mot de passe DB
REDIS_HOST	Oui	Endpoint Redis
MQ_HOST	Oui	Endpoint RabbitMQ / Amazon MQ
JWT_SECRET_ARN	Oui	Référence secret de signature JWT
BANK_API_BASE_URL	Oui	URL API système bancaire
SUPPLIER_API_BASE_URL	Oui	URL API fournisseur
LOG_LEVEL_ROOT	Non	Niveau de log global (INFO par défaut)

Les valeurs sensibles ne sont jamais stockées en clair dans le code ou dans les artefacts.

4.1.3 - Configuration applicative

La configuration est externalisée et pilotée par environnement.

Elle couvre notamment :

- **Paramètres Spring Boot ;**
- Configuration logs ;
- Paramètres de connexions DB/cache/messaging ;
- Paramètres d'intégration externe ;
- Paramètres de résilience (timeouts, retry, circuit breaker).



4.1.3.1 - Fichier *application.yml*

Fichier principal de configuration applicative (valeurs non sensibles), incluant :

- Paramètres serveur et profils ;
- Configuration des endpoints internes ;
- Réglages techniques généraux (pool, timeouts, etc.).

4.1.3.2 - Fichier *application-<env>.yml*

Surcharge spécifique par environnement (**application-dev.yml**, **application-test.yml**, **application-prod.yml**) :

- URLs de services ;
- Niveaux de logs ;
- Paramètres de performance adaptés au contexte.

4.1.3.3 - Fichier *logback-spring.xml*

Configuration de la journalisation :

- Format JSON ;
- Niveaux de logs ;
- Champs de corrélation ;
- Destination vers la collecte centralisée (**CloudWatch**).

4.1.4 - Déploiement

Le déploiement backend suit la séquence suivante :

- Sélectionner la version d'artefact à promouvoir ;
- Mettre à jour la **task definition ECS** (image + variables + secrets) ;
- Lancer le déploiement du **service ECS Fargate** ;
- Laisser l'**ALB router** progressivement vers les nouvelles tâches saines ;
- Exécuter les **migrations Flyway** si requises ;
- Valider le statut final du service (stable/running).

Le déploiement est automatisé via **GitHub Actions** et tracé dans l'historique **CI/CD**.



4.1.5 - Vérifications post-déploiement

Après déploiement, vérifier systématiquement :

- Etat healthy des tâches **ECS** ;
- Réussite des **health checks ALB** (/actuator/health) ;
- Accessibilité des **endpoints API** critiques ;
- Absence d'erreurs bloquantes dans **CloudWatch Logs** ;
- Latence et taux d'erreur dans les dashboards de supervision ;
- Connectivité aux dépendances (**RDS, Redis, MQ, services externes**).

En cas d'anomalie critique, appliquer la procédure de rollback définie en section 6.5.3.

4.1.6 - Guide pour le déploiement backend

Ce guide décrit la procédure détaillée de déploiement d'une version backend déjà construite sur l'infrastructure cible de **HandlePizza**.

Il s'applique à une solution dont les composants techniques sont déjà en place : **cluster ECS, service Fargate, ALB, target group, Secrets Manager, RDS MySQL, ElastiCache Redis, Amazon MQ et CloudWatch**.

Le backend **HandlePizza** est exécuté sous forme de **conteneurs Spring Boot sur Amazon ECS Fargate**.

Amazon ECS est le service AWS d'orchestration de conteneurs et **Fargate est son mode d'exécution managé**, qui permet de lancer des conteneurs sans administrer soi-même les serveurs hôtes.

Le service backend est exposé via **un Application Load Balancer (ALB)**, qui reçoit **les requêtes HTTP/HTTPS** et les répartit vers les tâches backend saines.

La configuration sensible est fournie par **AWS Secrets Manager, service de stockage sécurisé des secrets techniques**.

Les journaux d'exécution sont centralisés dans **Amazon CloudWatch Logs, service AWS de collecte et de supervision des logs et métriques**.



Etape 1 : identification de la version backend à déployer

Avant toute action, identifier précisément :

- L'environnement cible (**DEV, TEST-UAT ou PROD**),
- La version backend à promouvoir,
- Le tag de l'image conteneur correspondante,
- La présence éventuelle d'une migration **Flyway** associée à cette version.

Résultat attendu : la version applicative à déployer est connue sans ambiguïté, l'environnement cible est confirmé et la compatibilité avec le schéma de base de données est identifiée.

Etape 2 : vérification des accès nécessaires

Se connecter avec un compte disposant des habilitations nécessaires sur :

- **Amazon ECS,**
- **Elastic Load Balancing,**
- **AWS Secrets Manager,**
- **Amazon CloudWatch,**
- **Amazon RDS,**
- **Amazon ElastiCache,**
- **Amazon MQ,**
- **GitHub Actions** si le déploiement est lancé via **pipeline CI/CD**

Résultat attendu : l'exploitant peut consulter et piloter **le service ECS** et peut également vérifier les logs, la santé des tâches et les secrets utilisés.

Etape 3 : vérification de l'infrastructure cible

Avant de déployer une nouvelle version, contrôler la disponibilité des composants déjà provisionnés :

- **Le cluster ECS cible** existe et est accessible,
- **Le service ECS backend** existe,
- **L'ALB et son target group** sont actifs,
- **L'instance RDS MySQL** est en état Available,
- **Le cluster ElastiCache Redis** est disponible,



- **Le broker Amazon MQ / RabbitMQ** est disponible,
- Les secrets nécessaires existent dans **Secrets Manager**.

Résultat attendu : toutes les dépendances techniques critiques sont disponible avant le lancement du déploiement.

Étape 4 : vérification de la disponibilité de l'artefact à déployer

Le backend étant déjà développé, il existe sous forme d'artefact prêt à l'emploi :

- Image conteneurisée backend versionnée,
- **Task definition ECS associée** ou révision à mettre à jour,
- Paramètres d'environnement attendus,
- **Scripts Flyway** si la version impose une évolution de schéma.

Vérifier que l'image conteneurisée attendue est bien présente dans le registre utilisé par le projet et que son tag correspond à la version validée.

Résultat attendu : l'image backend exacte à déployer est disponible et la version ne nécessite pas de reconstruction.

Étape 5 : vérification de la Task Definition ECS

La task definition ECS est la définition technique d'exécution d'un **conteneur dans ECS**. Elle précise notamment :

- L'image à lancer,
- Le CPU et la mémoire allouée,
- Le port du conteneur,
- Les variables d'environnement,
- Les secrets injectés,
- La configuration de logs,
- Les paramètres réseau.

Ouvrir la **task definition backend** et vérifier :

- L'image conteneur cible,
- Le port applicatif du conteneur,
- La cohérence CPU/mémoire avec l'environnement,



- La présence des variables obligatoires,
- La présence des secrets obligatoires,
- La configuration **CloudWatch Logs**.

Résultat attendu : la future révision de **task definition** est complète et cohérente avec la version à déployer.

Étape 6 : vérification des variables d'environnement backend

Contrôler que les variables nécessaires sont bien définies pour l'environnement cible, notamment :

- **SPRING_PROFILES_ACTIVE**
- **SERVER_PORT**
- **DB_HOST**
- **DB_PORT**
- **DB_NAME**
- **REDIS_HOST**
- **MQ_HOST**
- **BANK_API_BASE_URL**
- **SUPPLIER_API_BASE_URL**
- **LOG_LEVEL_ROOT**

Ces variables permettent à **Spring Boot** de connaître son profil actif, ses connexions techniques et les URLs de ses dépendances externes.

Résultat attendu : aucune variable nécessaire au démarrage n'est absente et les valeurs correspondent à l'environnement cible.

Étape 7 : vérification des secrets dans AWS Secrets manager

AWS Secrets Manager est le **service AWS de stockage sécurisé des informations sensibles**, comme **les identifiants de base de données ou les secrets JWT**. Dans **HandlePizza**, ces secrets ne sont pas stockés en clair dans le code ni dans les artefacts ; ils sont injectés au démarrage **des tâches ECS**.

Il faut donc contrôler la présence et la validité des secrets suivants :

- **Identifiant DB,**



- **Mot de passe DB,**
- **Secret de signature JWT,**
- Eventuellement **clés d'API tierces** si gérées ainsi.

Point d'attention : selon la documentation AWS, lorsqu'un secret est injecté dans une tâche ECS comme variable d'environnement, sa nouvelle valeur n'est prise en compte qu'au démarrage d'une nouvelle tâche. Ainsi, si un secret a été modifié ou tourné, un redéploiement ou redémarrage des tâches est nécessaire.

Résultat attendu : les secrets existent, sont actifs et correspondent au bon environnement.

Étape 8 : vérification de la configuration de logs CloudWatch

Amazon CloudWatch Logs centralise les logs produits par **les conteneurs ECS**. La **task definition** doit contenir une configuration **awslogs**, qui permet à **Fargate** d'envoyer les journaux applicatifs vers le bon groupe de logs.

Vérifier dans **la task definition** :

- Le driver de log **awslogs**,
- Le nom du log group,
- La région AWS,
- Le préfixe de flux de logs.

Résultat attendu : les futures tâches backend publieront bien leurs logs dans **CloudWatch**.

Étape 9 : vérification de la configuration du health check

Le backend étant exposé via **un ALB**, celui-ci contrôle la santé des tâches grâce à **un target group health check**. Un health check est une requête automatique envoyée par le **load balancer** à intervalles réguliers pour savoir si une tâche est exploitable.

Vérifier :

- Le target group associé au service backend,
- Le protocole utilisé,
- Le port contrôlé,
- Le chemin de health check, par exemple /actuator/health,
- Les seuils de succès et d'échec.



Résultat attendu : le chemin de santé configuré dans l'ALB correspond à l'endpoint réellement exposé par l'application et une tâche démarrée peut devenir healthy si son démarrage se passe correctement.

Étape 10 : déclenchement du déploiement de la nouvelle version

Le déploiement backend consiste à demander au **service ECS** de lancer une nouvelle révision de tâche avec la nouvelle image et la nouvelle configuration.

Deux modes de lancement peuvent être utilisés.

Mode pipeline CI/CD :

- Ouvrir le **workflow GitHub Actions** de déploiement backend,
- Sélectionner l'environnement cible,
- Sélectionner la version à promouvoir,
- Lancer le workflow,
- Suivre l'exécution jusqu'à **la phase de mise à jour ECS**.

Mode console AWS :

- Ouvrir **Amazon ECS**,
- Sélectionner le cluster cible,
- Ouvrir le service backend,
- Choisir Update service,
- Sélectionner **la nouvelle révision de task definition**,
- Confirmer le déploiement.

Selon la documentation AWS, lors **d'un Update service**, **ECS** lance d'abord les nouvelles tâches puis retire progressivement les anciennes une fois les nouvelles considérées comme saines.

Résultat attendu : **un nouveau déploiement ECS** apparaît sur le service backend.

Étape 11 : suivi du cycle de démarrage des nouvelles tâches

Après déclenchement, suivre **les nouvelles tâches dans ECS** :

- Création,



- Etat **PENDING**,
- Téléchargement / initialisation du conteneur,
- Etat **RUNNING**.

Si une tâche s'arrête immédiatement, consulter sans attendre :

- Les événements du **service ECS**,
- **Les logs CloudWatch**,
- La configuration des secrets,
- La configuration réseau,
- **La connectivité DB / Redis / MQ**.

Résultat attendu : les nouvelles tâches démarrent et restent actives en état **RUNNING**.

Étape 12 : vérification de l'enregistrement des nouvelles tâches dans l'ALB

Lorsque les nouvelles tâches démarrent, elles sont enregistrées dans **le target group de l'ALB**. **L'ALB** ne leur envoie du trafic qu'une fois **les health checks validés**.

Dans **le target group**, vérifier :

- L'apparition des nouvelles cibles,
- Leur passage à l'état healthy,
- Le retrait progressif des anciennes cibles lors du rolling update.

Résultat attendu : le trafic API commence à basculer vers la nouvelle version sans coupure.

Étape 13 : exécution des migrations Flyway si la version l'exige

Flyway est l'outil de migration de schéma utilisé pour versionner et appliquer les évolutions de base de données. Si la version backend dépend d'un nouveau schéma, les migrations doivent être exécutées selon la procédure 4.3.

Vérifier :

- Si une migration est requise,
- Si la sauvegarde DB préalable a été réalisée,
- Si la migration a réussi sans erreur.



Résultat attendu : le schéma de base est compatible avec la nouvelle version applicative.

Étape 14 : vérification de la stabilité finale du service ECS

Une fois les nouvelles tâches démarrées, vérifier dans **ECS** :

- Que le service revient à un état stable,
- Que le nombre de tâches Running est conforme,
- Qu'aucun événement d'échec critique n'est remonté.

Résultat attendu : le service backend est stabilisé sur la nouvelle version.

Étape 15 : contrôle des logs dans CloudWatch

Ouvrir le **log group backend** dans **CloudWatch Logs** et vérifier :

- Le démarrage complet de **Spring Boot**,
- L'absence d'exception bloquante,
- La connexion réussie à **MySQL**,
- L'accès correct à **Redis**,
- L'accès correct à **RabbitMQ**,
- L'absence d'erreur de secret ou de configuration.

Résultat attendu : les logs montrent un démarrage nominal de l'application.

Étape 16 : vérification des endpoints backend critiques

Tester les endpoints minimaux nécessaires à la validation technique :

- Endpoint de santé,
- Endpoint d'authentification,
- Endpoint métier simple non destructif.

Contrôler :

- Le code HTTP,
- Le temps de réponse,
- L'absence d'erreur 5xx,
- La cohérence générale du comportement.



Résultat attendu : l'API backend répond correctement après déploiement.

Étape 17 : vérification de la connectivité applicative aux dépendances

Confirmer que le backend communique correctement avec ses dépendances :

- **RDS MySQL** pour la persistance métier,
- **ElastiCache Redis** pour le cache,
- **Amazon MQ / RabbitMQ** pour les échanges asynchrones,
- **Le système bancaire et la plateforme fournisseur** si un test contrôlé est prévu sur l'environnement cible.

Résultat attendu : aucune dépendance critique n'est bloquante après déploiement.

Étape 18 : vérification des métriques et des alertes de supervision

Dans **CloudWatch** ou le dashboard d'exploitation, contrôler :

- La latence des APIs,
- Le taux d'erreur,
- Le CPU et **mémoire des tâches ECS**,
- La saturation DB,
- Les anomalies sur **files RabbitMQ**,
- Les éventuelles alarmes actives.

Résultat attendu : aucune alerte critique ne subsiste après mise en service de la version.

Étape 19 : décision à propos de la validation ou du rollback

Valider le déploiement si l'ensemble des conditions suivantes sont réunies :

- **Tâches ECS stables**,
- **Cibles ALB healthy**,
- Logs applicatifs sans erreur critique,
- Endpoints API fonctionnels,
- Dépendances techniques accessibles,
- Aucune alerte critique persistante.

Déclencher un rollback si :



- Les tâches ne deviennent pas stables,
- **Les health checks** échouent,
- Les logs indiquent une erreur bloquante,
- La nouvelle version ne répond pas correctement aux contrôles minimaux.

Le rollback consiste à :

- Redéployer la précédente révision stable **de task definition**,
- Vérifier à nouveau **la santé ECS / ALB**,
- Refaire les contrôles minimaux de validation.

Résultat attendu : soit la nouvelle version est validée, soit le service est revenu à une version antérieure stable.

Étape 20 : traçage de l'opération dans le journal d'exploitation

Une fois l'opération terminée, enregistrer :

- La date et l'heure,
- L'environnement,
- La version déployée,
- Le nom de l'opérateur,
- La présence ou non **d'une migration Flyway**,
- Le résultat final,
- L'existence d'un rollback éventuel,
- Les incidents observés.

Résultat attendu : l'opération de déploiement est historisée, traçable et auditable.

4.1.6.1 - Informations opérationnelles à renseigner lors de la mise en œuvre

Afin de permettre une exploitation effective de la solution une fois l'infrastructure réellement déployée, les informations opérationnelles suivantes devront être renseignées pour chaque environnement (**DEV, TEST-UAT, PROD**). Cette partie constitue un support opérationnel à compléter lors de la phase de mise en œuvre effective de l'infrastructure et avant le passage en exploitation.



Ressources AWS de référence

- Région AWS utilisée,
- Nom du **cluster ECS**,
- Nom du **service ECS backend**,
- Nom de la famille **de la task definition**,
- Nom de **l'Application Load Balancer**,
- Nom du **target group backend**,
- Nom du **log group CloudWatch**,
- Références **des secrets AWS Secrets Manager** utilisés par l'application.

Chemins de navigation console / exploitation

- Accès au **service ECS backend**,
- Accès au **target group ALB**,
- Accès aux **logs CloudWatch**,
- Accès aux secrets applicatifs,
- Accès aux services de données (**RDS, ElastiCache, Amazon MQ**).

Valeurs attendues par environnement

- **Profil Spring actif**,
- Endpoints base de données, cache et messaging,
- URLs des services tiers,
- Paramètres techniques spécifiques à l'environnement.

Table de diagnostic rapide

- Erreurs de démarrage **des tâches ECS**,
- Échec de **health check ALB**,
- Problèmes de secrets,
- Défauts de **connectivité DB / Redis / MQ**,

Erreurs applicatives bloquantes relevées **dans CloudWatch**.



4.2 - Déploiement frontend (S3 + CloudFront + WAF)

Le frontend de **HandlePizza** est déployé sous forme d'assets statiques (**HTML/CSS/JS**) publiés dans **S3** et distribués via **CloudFront**.

La protection en bordure est assurée par **WAF**.

4.2.1 - Artefacts

Les artefacts frontend sont produits par la **pipeline CI/CD** après build (**Vite**), puis versionnés.

Contenu type :

- index.html
- **Bundles JavaScript/CSS** optimisés
- Assets statiques (images, polices, icônes)

Chaque livraison frontend est associée à une version traçable (commit/tag/release).

4.2.2 - Publication des assets

La publication suit les étapes suivantes :

- Build du frontend sur la version ciblée ;
- Dépôt des assets dans le **bucket S3** de l'environnement cible ;
- Application des politiques de cache (longue durée pour assets fingerprintés, plus courte pour index.html) ;
- Vérification des droits d'accès et **métadonnées HTTP**.

Objectif : garantir une diffusion correcte des fichiers statiques sans rupture utilisateur.

4.2.3 - Invalidation cache CloudFront

Après publication, une invalidation **CloudFront** est exécutée pour forcer la prise en compte des nouvelles versions.

Règles usuelles :

- Invalidation ciblée (/index.html) au minimum ;

DCAC
DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com
S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



- Invalidation plus large (/*) en cas de changement majeur ;
- Traçabilité de l'invalidation dans la pipeline.

Objectif : éviter la persistance de contenus obsolètes côté edge.

4.2.4 - Vérification post-déploiement

Après déploiement frontend, contrôler :

- **Accessibilité HTTPS** de l'interface ;
- Chargement correct des assets (pas d'erreurs 404/403/5xx) ;
- Cohérence de version affichée ;
- Bon fonctionnement des parcours critiques ;
- Absence d'**alertes WAF** anormales.

En cas d'échec, revenir à la version frontend précédente (rollback artefact) et réexécuter les contrôles.

4.2.5 - Guide pour le déploiement frontend

Ce guide décrit la procédure détaillée de déploiement d'une version frontend sur l'infrastructure cible de **HandlePizza**. Il s'applique à une solution dont les composants techniques sont déjà en place : **bucket Amazon S3, distribution Amazon CloudFront, protection AWS WAF, pipeline CI/CD et supervision associée.**

Le frontend de **HandlePizza** est distribué sous forme d'assets statiques (**HTML, CSS, JavaScript, images, polices**), générés à l'issue du build de l'application web.

Amazon S3 est le service AWS de stockage objet utilisé pour héberger les fichiers statiques du frontend. **Amazon CloudFront est le CDN AWS** chargé de distribuer ces fichiers au plus près des utilisateurs, via un réseau de cache mondial. **AWS WAF est le pare-feu applicatif** positionné en amont, utilisé pour filtrer certaines requêtes malveillantes ou non conformes.

Le déploiement frontend consiste donc à **publier une nouvelle version des fichiers statiques dans S3, puis à forcer si nécessaire la mise à jour du cache CloudFront** afin que les utilisateurs récupèrent la bonne version.



Étape 1 : identification de la version frontend à déployer

Avant toute action, identifier précisément :

- L'environnement cible (**DEV, TEST-UAT ou PROD**),
- La version frontend à promouvoir,
- Le package d'assets correspondant,
- La présence éventuelle de changements de configuration applicative liés à cette version.

Résultat attendu : la version frontend à déployer est connue sans ambiguïté, l'environnement cible est confirmé et l'artefact associé est identifié.

Étape 2 : vérification des accès nécessaires

Se connecter avec un compte disposant des habilitations nécessaires sur :

- **Amazon S3**,
- **Amazon CloudFront**,
- **AWS WAF**,
- **Amazon CloudWatch** si une supervision ou des métriques de distribution sont consultées,
- **GitHub Actions** si le déploiement est lancé **via pipeline CI/CD**.

Résultat attendu : l'exploitant peut publier les assets dans **S3**, déclencher ou vérifier l'invalidation **CloudFront** et contrôler l'état général de la distribution.

Étape 3 : vérification de l'infrastructure cible

Avant de déployer une nouvelle version, contrôler la disponibilité des composants déjà provisionnés :

- **Le bucket S3 frontend** existe et est accessible,
- **La distribution CloudFront** existe et est en état opérationnel,
- L'association entre **CloudFront et le bucket S3** est correcte,
- **La protection WAF** est bien rattachée à la distribution cible,
- Les éventuelles politiques de cache sont en place.



Résultat attendu : l'infrastructure de diffusion frontend est disponible avant le lancement du déploiement.

Étape 4 : vérification de la disponibilité de l'artefact à déployer

Si le frontend est déjà développé, il existe sous forme d'artefact prêt à l'emploi :

- **index.html**,
- **Bundles JavaScript versionnés**,
- **Fichiers CSS**,
- Images, icônes, polices et autres assets statiques.

Vérifier que l'artefact frontend attendu est bien présent dans le dépôt d'artefacts ou dans la sortie de pipeline correspondant à la version validée.

Résultat attendu : le package frontend exact à déployer est disponible et ne nécessite pas de reconstruction.

Étape 5 : vérification de la structure des assets à publier

Avant publication, contrôler :

- La présence du fichier **index.html**,
- La présence de tous **les bundles JavaScript et CSS attendus**,
- La cohérence des noms de fichiers versionnés ou fingerprintés,
- L'absence de fichiers manquants ou corrompus,
- La cohérence éventuelle des chemins de ressources.

Dans une architecture SPA, **index.html** constitue le point d'entrée principal de l'application web. Les autres fichiers statiques sont ensuite chargés par le navigateur pour afficher **l'interface React**.

Résultat attendu : les assets frontend sont complets, cohérents et prêts à être publiés.

Étape 6 : vérification de la configuration applicative frontend

Avant publication, vérifier que la configuration buildée du frontend correspond bien à l'environnement cible, notamment :



- URL des APIs backend,
- Paramètres de mode (**DEV, TEST-UAT, PROD**),
- Éventuelles variables de build exposées au frontend,
- Configuration de routage ou de base path si applicable.

Point d'attention : une mauvaise configuration frontend peut conduire à une application qui se charge visuellement, mais qui appelle les mauvais endpoints backend.

Résultat attendu : la version frontend à publier est configurée pour le bon environnement.

Étape 7 : vérification des métadonnées de publication

Avant dépôt dans **S3**, vérifier si **les métadonnées HTTP** attendues sont correctement préparées, notamment :

- **Type MIME** des fichiers ;
- Politique de cache ;
- Encodage éventuel ;
- Séparation entre fichiers fortement cachables et fichiers à rafraîchir rapidement.

En pratique :

- **index.html** doit généralement avoir une politique de cache courte,
- Les bundles fingerprintés peuvent être conservés plus longtemps en cache.

Résultat attendu : les fichiers frontend sont prêts à être publiés avec des métadonnées cohérentes.

Étape 8 : déclenchement du déploiement frontend

Le déploiement frontend consiste à publier les nouveaux assets dans **le bucket S3** cible. Deux modes de lancement peuvent être utilisés.

Mode pipeline CI/CD :

- Ouvrir **le workflow GitHub Actions** de déploiement frontend ;
- Sélectionner l'environnement cible ;
- Sélectionner la version à promouvoir ;
- Lancer le workflow ;



- Suivre l'exécution jusqu'à **la phase de publication S3**.

Mode opérationnel direct :

- Ouvrir l'accès **au bucket S3** cible ;
- Déposer les nouveaux assets dans l'emplacement prévu ;
- Remplacer les fichiers obsolètes selon la stratégie de publication définie ;
- Vérifier que les objets attendus sont bien présents dans le bucket.

Résultat attendu : la nouvelle version des assets frontend est publiée dans **le bucket S3** cible.

Étape 9 : vérification de la publication dans S3

Après publication, contrôler dans **S3** :

- La présence du nouvel **index.html** ;
- La présence des bundles et assets associés ;
- La date de mise à jour des objets ;
- La cohérence des noms de fichiers ;
- Les métadonnées principales si elles sont visibles ou contrôlées.

Résultat attendu : tous les fichiers attendus sont présents dans **S3** dans leur version cible.

Étape 10 : vérification de la distribution CloudFront

CloudFront sert les fichiers présents dans **S3**, mais peut continuer temporairement à fournir une version mise en cache. Il faut donc vérifier :

- Que la distribution cible est active,
- Qu'elle pointe bien vers le bon bucket d'origine,
- Qu'aucune erreur de configuration n'empêche la diffusion.

Résultat attendu : la **distribution CloudFront** est prête à exposer la nouvelle version frontend.

Étape 11 : exécution de l'invalidation CloudFront



Une **invalidation CloudFront** permet de demander au **CDN** d'abandonner certaines versions en cache pour recharger les nouvelles depuis **S3**. Cette opération est particulièrement importante pour **index.html**, qui pilote le chargement des bundles.

Déclencher une invalidation selon la stratégie retenue :

- Invalidation minimale sur **/index.html**,
- Invalidation élargie sur **/*** en cas de changement plus global,
- Invalidation complémentaire sur certains chemins si nécessaire.

Résultat attendu : **CloudFront** prend en compte la nouvelle version publiée et limite le risque de diffusion d'un frontend obsolète.

Étape 12 : vérification de l'état de l'invalidation

Suivre l'avancement de **l'invalidation CloudFront** et vérifier :

- Qu'elle est bien enregistrée,
- Qu'elle progresse normalement,
- Qu'elle atteint l'état Completed.

Résultat attendu : **l'invalidation CloudFront** est finalisée et la distribution peut servir la nouvelle version.

Étape 13 : vérification du comportement WAF

AWS WAF protège **la distribution CloudFront** contre certains trafics ou patterns considérés comme suspects. Après déploiement, vérifier :

- Qu'aucune **règle WAF** ne bloque anormalement l'accès au frontend,
- Qu'aucune hausse d'événements bloqués n'est observée sans raison,
- Que la nouvelle version ne déclenche pas de faux positifs évidents.

Résultat attendu : la nouvelle version frontend reste accessible normalement tout en conservant **la protection WAF**.

Étape 14 : contrôle de l'accessibilité du frontend

Tester l'accès au frontend via **l'URL HTTPS** de l'environnement cible et vérifier :



- Le chargement de la page,
- L'absence d'erreurs de chargement majeures,
- La présence des styles,
- Le bon chargement des scripts principaux,
- L'absence **d'erreurs 403, 404 ou 5xx** sur les assets critiques.

Résultat attendu : l'interface frontend s'affiche correctement via **CloudFront en HTTPS**.

Étape 15 : contrôle des parcours fonctionnels minimaux

Une fois le frontend accessible, vérifier quelques parcours minimaux :

- Ouverture de la page d'accueil,
- Navigation sur les principales routes,
- Chargement des données de base depuis le backend,
- Absence d'erreur de configuration visible,
- Bon comportement général de **la SPA**.

Résultat attendu : le frontend est techniquement accessible et fonctionnel sur les parcours de base.

Étape 16 : contrôle de la cohérence frontend / backend

Le frontend dépend du backend pour ses appels API. Il faut donc vérifier :

- Que les appels du frontend visent bien l'environnement backend attendu,
- Qu'aucune erreur **CORS** ou erreur d'URL n'apparaît,
- Que les premières réponses API sont conformes
- Que les parcours critiques ne sont pas bloqués par une incompatibilité de version.

Résultat attendu : le frontend et le backend sont compatibles dans l'environnement cible.

Étape 17 : contrôle des journaux et métriques de diffusion

Selon les moyens de supervision en place, contrôler :

- Les éventuelles erreurs de diffusion,
- Les taux **d'erreur HTTP** côté distribution,
- Les alertes éventuelles liées à **CloudFront, WAF** ou à l'accessibilité du frontend,



- Les erreurs visibles côté navigateur si une vérification applicative est réalisée.

Résultat attendu : aucune anomalie critique de diffusion ou d'accès n'est détectée après publication.

Étape 18 : décision à propos de la validation ou du rollback

Valider le déploiement si l'ensemble des conditions suivantes sont réunies :

- Assets publiés correctement dans **S3**,
- **Invalidation CloudFront** terminée,
- Frontend accessible en **HTTPS**,
- Parcours minimaux fonctionnels,
- Absence d'erreur critique de diffusion ou de configuration.

Déclencher un rollback si :

- Les assets publiés sont incomplets ou incorrects,
- L'interface ne se charge pas correctement,
- Les appels API sont incompatibles ou erronés,
- Une anomalie critique rend l'environnement inutilisable.

Le rollback consiste à :

- Republier la version frontend précédente dans **S3**,
- Relancer **l'invalidation CloudFront**,
- Refaire les contrôles minimaux de validation.

Résultat attendu : soit la nouvelle version frontend est validée, soit la version précédente est rétablie.

Étape 19 : traçage de l'opération dans le journal d'exploitation

Une fois l'opération terminée, enregistrer :

- La date et l'heure,
- L'environnement,
- La version frontend déployée,
- Le nom de l'opérateur,



- Le type d'invalidation exécutée,
- Le résultat final,
- L'existence éventuelle d'un rollback,
- Les incidents observés.

Résultat attendu : l'opération de déploiement frontend est historisée, traçable et auditable.

4.2.5.1 - Informations opérationnelles à renseigner lors de la mise en œuvre

Afin de permettre une exploitation effective de la solution une fois l'infrastructure réellement déployée, les informations opérationnelles suivantes devront être renseignées pour chaque environnement (**DEV, TEST-UAT, PROD**). Cet encadré constitue un support opérationnel à compléter lors de la phase de mise en œuvre effective de l'infrastructure et avant le passage en exploitation.

Ressources AWS de référence

- Région **AWS** utilisée,
- Nom du **bucket S3** frontend,
- Nom ou identifiant de la **distribution CloudFront**,
- Nom ou identifiant du **Web ACL WAF associé**,
- Éventuels chemins ou préfixes de publication,
- URL publique frontend de l'environnement.

Chemins de navigation console / exploitation

- Accès au **bucket S3** frontend,
- Accès à la **distribution CloudFront**,
- Accès à la **configuration WAF**,
- Accès aux éventuels dashboards ou journaux de supervision.

Valeurs attendues par environnement

- URL frontend publique,
- URL backend appelée par le frontend,
- Paramètres de cache
- Règles particulières de diffusion
- Éventuelles options spécifiques de build ou de routage.



Table de diagnostic rapide

- Assets absents ou incomplets dans **S3**,
- **Invalidation CloudFront** incomplète ou non exécutée,
- Frontend obsolète toujours servi au navigateur,
- Erreur **d'appel API** depuis le frontend,
- Blocage inattendu par **WAF**,
- **Erreur 403, 404 ou 5xx** sur les ressources statiques.

4.3 - Migrations base de données (Flyway)

Les évolutions de schéma de base de données sont gérées via **Flyway** et exécutées dans la chaîne de déploiement.

4.3.1 - Pré-requis

Avant exécution des migrations :

- Version applicative validée ;
- **Scripts Flyway** présents et versionnés ;
- **Accès DB** actif sur l'environnement cible ;
- **Sauvegarde DB** récente disponible ;
- Fenêtre de déploiement confirmée.

4.3.2 - Exécution

L'exécution suit la séquence suivante :

- Sélectionner l'environnement cible ;
- Exécuter **Flyway** sur les scripts non appliqués ;
- Vérifier le statut de migration (success/failure) ;
- Poursuivre le déploiement applicatif si la migration est validée.

Les scripts sont appliqués de manière ordonnée et traçable.

4.3.3 - Contrôles

Contrôles post-migration :

DCAC	100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com
DCAC.COM	S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



- Cohérence de la table d'historique **Flyway** ;
- Présence attendue des **nouveaux objets DB** ;
- Absence d'**erreurs SQL bloquantes** ;
- Validation fonctionnelle minimale sur parcours impactés.

Objectif : garantir la compatibilité entre schéma cible et version applicative déployée.

4.3.4 - Rollback / reprise

En cas d'échec migration :

- Interrompre la promotion vers l'environnement supérieur ;
- Appliquer la procédure de reprise documentée (correction script, relance contrôlée) ;
- Si nécessaire, restaurer la base depuis sauvegarde selon procédure de restauration ;
- Redéployer une version applicative compatible avec l'**état DB** restauré.

Le **rollback DB** est piloté avec prudence et toujours tracé dans les journaux d'exploitation.



5 - PROCÉDURE DE DÉMARRAGE / ARRÊT

Cette section décrit la procédure opérationnelle de démarrage et d'arrêt de **HandlePizza**, en distinguant la couche applicative, la couche données et les contrôles de santé associés.

L'objectif est de garantir des opérations maîtrisées, traçables et sans impact non contrôlé sur les utilisateurs.

5.1 - Couche application (ECS)

La couche applicative est déployée sur **ECS Fargate** et pilotée via les **services ECS**.

Démarrage :

- Vérifier la disponibilité des dépendances critiques (**RDS, Redis, RabbitMQ, Secrets Manager**).
- Vérifier la configuration cible (task definition, variables d'environnement, secrets).
- Positionner le **service ECS** sur le nombre de tâches souhaité (desired count > 0).
- Contrôler la montée progressive des tâches en état RUNNING puis HEALTHY.
- Vérifier la bonne prise en compte par l'**ALB** (targets saines).

Arrêt :

- Désactiver les déclenchements de déploiement automatique si nécessaire (fenêtre de maintenance).
- Réduire progressivement le desired count du **service ECS** jusqu'à 0.
- Vérifier le drainage du **trafic ALB** (plus de requêtes vers les tâches arrêtées).
- Contrôler l'absence d'erreurs anormales dans **CloudWatch Logs** pendant la phase d'arrêt.

Précaution :

- L'arrêt applicatif doit être réalisé hors pic d'activité et communiqué aux parties prenantes si impact utilisateur attendu.



5.2 - Couche données (RDS, Redis, RabbitMQ)

La couche de données est constituée de services managés en sous-réseau privé data.

Démarrage :

- Vérifier l'état Available de **RDS MySQL**.
- Vérifier la disponibilité d'**ElastiCache Redis**.
- Vérifier la disponibilité d'**Amazon MQ (RabbitMQ)** et des files/queues nécessaires.
- Confirmer la connectivité réseau depuis la couche applicative (security groups, routes, DNS internes).

Arrêt :

- En exploitation standard, les services data managés ne sont pas arrêtés systématiquement.
- En cas d'arrêt planifié exceptionnel :
 - o Arrêter d'abord la couche applicative (pour éviter les erreurs d'écriture/lecture).
 - o Appliquer les procédures spécifiques AWS du service concerné (RDS/Redis/MQ).
 - o Vérifier sauvegardes et points de restauration avant toute opération intrusive.

Précaution : Toute action d'arrêt sur la couche données doit être validée, tracée et couverte par une stratégie de reprise.

5.3 - Vérification de santé

Après tout démarrage (ou redémarrage), les contrôles suivants sont obligatoires :

Application :

- Endpoints de santé backend (/actuator/health) au statut attendu ;
- **Tâches ECS** en état stable ;
- Cibles **ALB healthy**.

Données et messaging :

- **Connexion DB** opérationnelle ;
- **Accès Redis** valide ;



- Publication/consommation RabbitMQ fonctionnelle.

Flux fonctionnels critiques :

- Appel API de test (authentification + endpoint métier simple) ;
- Vérification absence d'erreurs bloquantes dans CloudWatch Logs ;
- Contrôle métriques clés (latence, taux d'erreur, saturation ressources).

Critère de validation :

La plateforme est considérée opérationnelle lorsque les contrôles techniques et fonctionnels minimaux sont conformes, sans alerte critique active.



6 - PROCEDURE DE MISE A JOUR

Cette section décrit la procédure de mise à jour de **HandlePizza** pour les composants applicatifs, la base de données et la configuration technique.

L'objectif est de sécuriser les évolutions en limitant les risques de régression et en garantissant la traçabilité des changements.

6.1 - Mise à jour applicative

La mise à jour applicative couvre le backend (**ECS Fargate**) et le frontend (**S3/CloudFront**).

Séquence recommandée :

- Vérifier les prérequis de release (tests validés, artefacts versionnés, approbation de promotion).
- Déployer la nouvelle version backend via mise à jour de **task definition ECS**.
- Vérifier stabilité backend (tasks healthy, **health checks ALB**, logs sans erreur critique).
- Déployer les nouveaux assets frontend sur **S3**.
- Exécuter **l'invalidation CloudFront** (au minimum /index.html).
- Vérifier les parcours fonctionnels critiques.

Principes :

- Promotion d'artefacts immuables (pas de reconstruction entre environnements).
- Déploiement traçable via pipeline **CI/CD**.
- Rollback immédiat possible vers la version précédente en cas d'incident.

6.2 - Mise à jour base de données

Les évolutions de schéma sont versionnées et exécutées avec **Flyway**.

Séquence recommandée :

- Vérifier la présence des **scripts Flyway** versionnés et validés.
- Confirmer la disponibilité d'une **sauvegarde récente RDS**.
- Exécuter les migrations sur l'environnement cible.



- Contrôler **la table d'historique Flyway** et la cohérence des objets créés/modifiés.
- Valider la compatibilité applicative avec le schéma mis à jour.

Principes :

- Exécution ordonnée, non destructive et traçable.
- Fenêtre de déploiement maîtrisée pour les changements sensibles.
- Procédure de reprise documentée en cas d'échec de migration.

6.3 - Mise à jour configuration / secrets

La configuration et les secrets sont mis à jour sans modification du code source.

Configuration applicative :

- Mettre à jour les paramètres d'environnement (profil, endpoints, seuils, timeouts).
- Appliquer les changements dans les fichiers/profils concernés (application-<env>.yaml, paramètres runtime).
- Vérifier cohérence inter-environnements (**DEV/TEST-UAT/PROD**).

Secrets :

- Mettre à jour les secrets dans **AWS Secrets Manager (DB, JWT, clés API externes)**.
- Vérifier les **permissions IAM** des services consommateurs.
- Planifier/contrôler la rotation des secrets selon la politique sécurité.
- Tracer les changements (date, auteur, environnement, périmètre).

6.4 - Contrôles post-mise à jour

Après toute mise à jour, les contrôles suivants sont obligatoires :

Technique :

- **Services ECS** stables et healthy ;
- **Health checks ALB** valides ;
- **Connectivité RDS/Redis/RabbitMQ** opérationnelle ;
- Absence d'erreurs critiques dans **CloudWatch Logs**.

Fonctionnel :



- Vérification des parcours critiques (authentification, commande, paiement, suivi statut) ;
- Vérification des intégrations externes (banque/fournisseur) sur scénario nominal.

Supervision :

- Contrôle métriques clés (latence, taux d'erreur, saturation) ;
- Vérification absence d'alertes critiques post-déploiement ;
- Validation de la version effectivement active.

Critère de clôture :

La mise à jour est validée lorsque tous les contrôles techniques et fonctionnels sont conformes et qu'aucune alerte critique persistante n'est détectée.

6.5 - Guide pour la mise à jour de la solution

Ce guide décrit la procédure détaillée de mise à jour de **HandlePizza** sur une infrastructure déjà déployée et en exploitation. Il couvre la mise à jour des composants applicatifs, des éventuelles évolutions de base de données ainsi que des paramètres de configuration et des secrets techniques.

Une mise à jour peut concerner :

- Le backend déployé sur **Amazon ECS Fargate**,
- Le frontend distribué via **Amazon S3 et Amazon CloudFront**,
- Le schéma de base de données versionné avec **Flyway**,
- Les paramètres d'environnement et secrets stockés dans **AWS Secrets Manager**.

L'objectif de cette procédure est de garantir une évolution maîtrisée de la solution, sans reconstruction non contrôlée, avec validation, supervision et possibilité de retour arrière en cas d'incident.

Étape 1 : identification du périmètre de mise à jour

Avant toute action, identifier précisément :

- L'environnement cible (**DEV, TEST-UAT, PROD**),
- La version applicative à promouvoir,



- Les composants concernés par la mise à jour (backend, frontend, base de données, configuration, secrets),
- La présence ou non d'une **migration Flyway** associée,
- La nécessité éventuelle d'une fenêtre de maintenance.

Résultat attendu : le périmètre exact de la mise à jour est connu, ainsi que ses dépendances techniques et son impact potentiel.

Étape 2 : vérification des prérequis de release

Avant la mise à jour, vérifier :

- Que la version candidate a été validée par les tests attendus,
- Que les artefacts sont versionnés et disponibles,
- Que la promotion vers l'environnement cible a été autorisée,
- Que la version précédente stable reste disponible pour un éventuel rollback.

Dans **HandlePizza**, les mises à jour reposent sur le principe d'artefacts immuables : une version promue dans un environnement n'est pas reconstruite entre les environnements.

Résultat attendu : la version à mettre à jour est prête à être déployée dans des conditions maîtrisées.

Étape 3 : vérification des accès nécessaires

Se connecter avec un compte disposant des habilitations nécessaires sur :

- **Amazon ECS,**
- **Elastic Load Balancing,**
- **Amazon S3,**
- **Amazon CloudFront,**
- **AWS Secrets Manager,**
- **Amazon RDS,**
- **Amazon CloudWatch,**
- **Amazon ElastiCache,**
- **Amazon MQ,**
- **GitHub Actions** si la mise à jour est pilotée par **CI/CD**.



Résultat attendu : l'exploitant dispose des accès nécessaires pour appliquer, vérifier et tracer la mise à jour.

Étape 4 : vérification de l'état initial de la plateforme

Avant mise à jour, contrôler l'état général de la plateforme :

- **Service backend ECS** stable,
- **Cibles ALB** healthy,
- Frontend accessible,
- **Base RDS** disponible,
- **Redis** disponible,
- **RabbitMQ / Amazon MQ** disponible,
- Absence d'alerte critique active dans la supervision.

Une mise à jour ne doit pas être engagée sur une plateforme déjà dégradée sans analyse préalable.

Résultat attendu : la plateforme est dans un état nominal avant lancement de la mise à jour.

Étape 5 : vérification de la sauvegarde disponible

Si la mise à jour comporte un impact sur la base de données ou sur des éléments critiques de configuration, vérifier :

- La disponibilité d'une sauvegarde récente de **la base RDS MySQL**,
- La possibilité de restaurer la version précédente si nécessaire,
- La compatibilité entre version applicative cible et schéma attendu.

Résultat attendu : la capacité de reprise est confirmée avant toute opération sensible.

Étape 6 : mise à jour du backend applicatif

Si la mise à jour concerne le backend :

- Sélectionner la nouvelle version de l'image backend,
- Mettre à jour **la task definition ECS** si nécessaire,
- Lancer la mise à jour du **service ECS Fargate** selon la procédure de déploiement backend,



- Suivre le lancement des nouvelles tâches,
- Vérifier leur enregistrement dans **l'ALB**,
- Contrôler la stabilité finale du service.

Amazon ECS Fargate permet de remplacer progressivement les tâches anciennes par les nouvelles, sans administrer les serveurs sous-jacents tandis qu'**ALB** route le trafic uniquement vers les tâches considérées comme saines par **le health check**.

Résultat attendu : le backend est mis à jour sur la nouvelle version et revient à un état stable.

Étape 7 : mise à jour du frontend applicatif

Si la mise à jour concerne le frontend :

- Publier les nouveaux assets dans le **bucket S3** de l'environnement cible,
- Vérifier la présence des fichiers attendus,
- Déclencher une **invalidation CloudFront**,
- Attendre la prise en compte de l'invalidation,
- Vérifier **l'accessibilité HTTPS** de l'interface.

Amazon S3 héberge les fichiers statiques du frontend, **Amazon CloudFront** distribue ces fichiers via cache edge et une invalidation **CloudFront** permet de forcer le **CDN** à servir la nouvelle version.

Résultat attendu : le frontend exposé aux utilisateurs correspond à la nouvelle version attendue.

Étape 8 : mise à jour de la base de données si nécessaire

Si la version impose une évolution du schéma :

- Vérifier la présence des **scripts Flyway** versionnés,
- Exécuter les migrations dans l'environnement cible,
- Contrôler **la table d'historique Flyway**,
- Vérifier la présence et la cohérence des objets créés ou modifiés,
- Valider la compatibilité avec la version applicative déployée.

Flyway est l'outil de migration utilisé pour versionner et appliquer de manière ordonnée les évolutions de schéma.



Résultat attendu : la base de données est alignée avec la nouvelle version applicative.

Étape 9 : mise à jour de la configuration applicative si nécessaire

Si la mise à jour implique des changements de paramètres :

- Mettre à jour les variables d'environnement concernées,
- Mettre à jour les profils applicatifs ou paramètres runtime,
- Vérifier la cohérence entre environnement, version et configuration active,
- Redéployer les tâches si la prise en compte nécessite un redémarrage.

Résultat attendu : la configuration active est cohérente avec la version déployée.

Étape 10 : mise à jour des secrets si nécessaire

Si la mise à jour concerne des secrets techniques :

- Mettre à jour les valeurs dans **AWS Secrets Manager**,
- Vérifier **les permissions IAM** des services consommateurs,
- Confirmer que les nouvelles tâches relisent bien les secrets au démarrage,
- Redéployer ou redémarrer les tâches si nécessaire.

Point d'attention : lorsqu'un secret injecté dans **ECS** change, la nouvelle valeur n'est pas automatiquement visible dans une tâche déjà démarrée. Une relance des tâches est nécessaire pour prise en compte.

Résultat attendu : les secrets actifs sont à jour et correctement consommés par les services.

Étape 11 : contrôle technique post-mise à jour

Après mise à jour, vérifier obligatoirement :

- **Services ECS** stables et healthy,
- **Health checks ALB** valides,
- **Connectivité RDS, Redis, RabbitMQ** opérationnelle,
- Absence d'erreurs critiques dans **CloudWatch Logs**,
- Absence d'arrêt anormal de **tâches ECS**.

Résultat attendu : la plateforme reste techniquement stable après la mise à jour.



Étape 12 : contrôle fonctionnel post-mise à jour

Réaliser un contrôle fonctionnel minimal sur les parcours critiques :

- Authentification,
- Consultation d'un parcours métier simple,
- Création ou consultation de commande selon l'environnement,
- Enchaînement nominal d'un scénario clé,
- Vérification des intégrations externes sur scénario de test ou nominal selon le contexte.

Résultat attendu : les parcours majeurs restent conformes après mise à jour.

Étape 13 : contrôle de supervision post-mise à jour

Dans **CloudWatch** ou le tableau de bord d'exploitation, vérifier :

- La latence API,
- Le taux d'erreur,
- La charge CPU / mémoire des **tâches ECS**,
- La saturation éventuelle de la base de données,
- Les anomalies sur files de messages,
- L'absence d'alerte critique persistante.

Résultat attendu : aucun indicateur ne montre de dégradation anormale liée à la mise à jour.

Étape 14 : validation de la version active

Confirmer que la version réellement servie correspond bien à la version cible :

- Côté backend,
- Côté frontend,
- Côté schéma DB si applicable,
- Côté configuration si des changements étaient attendus.

Résultat attendu : l'environnement cible exécute bien la version officiellement promue.

Étape 15 : décision de validation ou de rollback



Valider la mise à jour si :

- Les contrôles techniques sont conformes,
- Les contrôles fonctionnels sont conformes,
- Aucune alerte critique persistante n'est détectée,
- La version active constatée est correcte.

Déclencher un rollback si :

- Le backend ou le frontend ne reviennent pas à un état nominal,
- Une migration entraîne une incompatibilité ou un dysfonctionnement bloquant,
- Une erreur critique apparaît dans les logs ou la supervision,
- Un parcours critique métier n'est plus opérationnel.

Le rollback consiste à :

- Redéployer la version applicative précédente,
- Rétablir si nécessaire les assets frontend précédents,
- Restaurer la configuration précédente,
- Appliquer la procédure de reprise DB si un retour arrière de la base est requis.

Résultat attendu : soit la nouvelle version est validée, soit la plateforme revient à une version antérieure stable.

Étape 16 : traçage de l'opération de mise à jour

Une fois l'opération terminée, enregistrer dans le journal d'exploitation :

- La date et l'heure,
- L'environnement,
- La version précédente,
- La version mise à jour,
- Les composants concernés,
- La présence ou non **d'une migration Flyway**,
- Le nom de l'opérateur,
- Le résultat final,
- La présence éventuelle d'un rollback,
- Les incidents observés.



Résultat attendu : la mise à jour est historisée, traçable et auditable.

6.5.1 - Informations opérationnelles à renseigner lors de la mise en œuvre

Afin de permettre une exploitation effective de la solution une fois l'infrastructure réellement déployée, les informations opérationnelles suivantes devront être renseignées pour chaque environnement (**DEV, TEST-UAT, PROD**). Cet encadré constitue un support opérationnel à compléter lors de la phase de mise en œuvre effective de l'infrastructure et avant le passage en exploitation.

Ressources techniques de référence

- Région AWS utilisée,
- Nom du **cluster ECS**,
- Nom du service backend,
- Nom de la famille de **la task definition**,
- Nom du **bucket S3** frontend,
- Identifiant de la **distribution CloudFront**,
- Nom **de l'ALB** et du target group,
- Nom **du log group CloudWatch**,
- Références des secrets applicatifs,
- Référence de **la base RDS**,
- Référence **du cluster Redis**,
- Référence **du broker Amazon MQ**.

Chemins d'exploitation

- Accès au **service ECS**,
- Accès au **bucket S3**,
- Accès à **CloudFront**,
- Accès au **target group ALB**,
- Accès à **Secrets Manager**,
- Accès **aux logs CloudWatch**,
- Accès à **RDS, ElastiCache et Amazon MQ**,
- Accès au **pipeline GitHub Actions**.

Valeurs attendues par environnement

- Version backend active,



- Version frontend active,
- **Profil Spring actif**,
- Endpoint de dépendances,
- Paramètres critiques de configuration,
- URL des services tiers,
- Stratégie d'invalidation frontend,
- Indicateurs de santé attendus.

Table de diagnostic rapide

- Échec de **démarrage ECS**,
- Tâche backend non healthy,
- **Invalidation CloudFront** non finalisée,
- Erreur de configuration ou de secret,
- Incompatibilité version applicative / schéma DB,
- Erreur API ou erreur frontend post-mise à jour,
- Alerte **critique CloudWatch** après release.



7 - SUPERVISION ET MONITORING

Cette section décrit les mécanismes de supervision de **HandlePizza** pour garantir la disponibilité du service, détecter rapidement les anomalies et faciliter le diagnostic en exploitation.

7.1 - Logs centralisés (CloudWatch)

Les journaux applicatifs et techniques sont centralisés dans **CloudWatch Logs**.

Les applications publient des logs structurés (**format JSON**) avec horodatage, niveau de sévérité et identifiants de corrélation.

Contrôles d'exploitation recommandés :

- Vérifier la présence continue des flux de logs backend ;
- Vérifier la politique de rétention selon environnement (**DEV/TEST-UAT/PROD**).
- Contrôler l'absence de pics **d'erreurs ERROR/WARN** anormaux ;
- Rechercher les exceptions critiques sur les parcours métier sensibles ;
- Corréler les logs avec l'heure des incidents remontés ;

Vérifier la politique de rétention selon environnement (**DEV/TEST-UAT/PROD**).

Objectif : disposer d'une base fiable pour diagnostic, audit et post-mortem incident.

7.2 - Métriques et alertes

La supervision métrique couvre les indicateurs clés de performance et de stabilité de la plateforme.

Métriques principales :

- Disponibilité des **services ECS/ALB** ;
- **Latence API (p50/p95/p99** si disponible) ;
- **Taux d'erreurs HTTP** (4xx/5xx) ;
- CPU/mémoire des tâches backend ;
- Saturation des **connexions DB / latence RDS** ;
- État des **files MQ** et consommation.



Alerting :

- Seuils d'alerte définis par criticité (warning/critical) ;
- Notification vers les canaux d'astreinte/exploitation ;
- Escalade automatique si incident non résolu dans le délai attendu.

Objectif : détecter proactivement les dérives avant impact utilisateur majeur.

7.3 - Supervision des integrations externes

Les appels vers les services bancaires et fournisseur sont supervisés en continu.

Indicateurs suivis :

- Latence des appels sortants ;
- Taux d'échec et timeouts ;
- Volume de retries ;
- Fréquence d'ouverture des circuit breakers ;
- Disponibilité perçue des partenaires externes.

Contrôles :

- Vérifier la cohérence entre erreurs techniques et statuts métier ;
- Déclencher alerte en cas de dégradation persistante ;
- Isoler rapidement un flux externe défaillant pour préserver le cœur applicatif.

Objectif : maintenir un service stable malgré la dépendance à des systèmes tiers.

7.4 - Gestion d'incident (runbook)

La gestion d'incident est pilotée via un **runbook opérationnel**.

Étapes minimales :

- Qualification (impact, criticité, périmètre) ;
- Diagnostic initial (logs, métriques, traces, dépendances) ;
- Actions de remédiation standard (redémarrage, rollback, limitation de flux) ;
- Escalade vers les rôles techniques/métier ;
- Communication de statut jusqu'à résolution ;



- Clôture avec retour d'expérience (**RCA et actions préventives**).

Objectif : réduire **le MTTR** et homogénéiser la réponse opérationnelle.



8 - PROCÉDURE DE SAUVEGARDE ET RESTAURATION

Cette section formalise la stratégie de protection des données, de restauration et de vérification périodique pour garantir la continuité d'activité.

8.1 - Politique de sauvegarde

La politique de sauvegarde couvre en priorité les données métier critiques (**RDS MySQL**).

Principes retenus :

- Sauvegardes automatiques planifiées ;
- Politique de rétention définie par environnement ;
- Chiffrement des sauvegardes au repos ;
- Traçabilité des opérations de sauvegarde ;
- Vérification régulière de la complétude des sauvegardes.

Prérequis organisationnels :

- Définir **RPO/RTO** cibles ;
- Documenter fenêtres de sauvegarde et responsabilités d'exploitation.

8.2 - Procédure de restauration

En cas d'incident de données, la restauration suit une procédure contrôlée.

Séquence recommandée :

- Qualifier l'incident et décider la stratégie de restauration ;
- Identifier le point de restauration cible ;
- Restaurer la base sur instance cible (ou instance temporaire de validation) ;
- Vérifier l'intégrité des données restaurées ;
- Reconnecter l'application de manière contrôlée ;
- Exécuter les contrôles techniques et fonctionnels post-restauration ;
- Tracer l'opération (date, version, opérateur, périmètre).



Précaution : toute restauration en production doit être validée et communiquée avant exécution.

8.3 - Tests périodiques de restauration

Les restaurations doivent être testées régulièrement pour valider l'opérabilité réelle du dispositif de secours.

Principes retenus :

- Planification de tests de restauration à fréquence définie ;
- Exécution sur environnement contrôlé ;
- Mesure des temps de reprise (**RTO réel**) ;
- Vérification de cohérence fonctionnelle des données restaurées ;
- Capitalisation des résultats et actions d'amélioration.

Objectif : garantir que la restauration est réellement exécutable dans les délais attendus en cas de crise.

8.4 - Guide pour la sauvegarde et la restauration

Ce guide décrit la procédure opérationnelle de sauvegarde et de restauration des données critiques de **HandlePizza** sur une infrastructure déjà déployée et en exploitation. La priorité porte sur la base de données métier **Amazon RDS MySQL**, qui stocke les informations structurantes du système : commandes, paiements, stocks, réapprovisionnements, référentiels et éléments d'audit.

Amazon RDS est le service AWS de base de données relationnelle managée. Dans le cadre de **HandlePizza**, il permet de bénéficier de sauvegardes automatiques, de mécanismes de restauration contrôlée et d'une supervision intégrée, sans administrer directement le moteur et l'infrastructure sous-jacente.

La sauvegarde a pour objectif de protéger les données contre les pertes, corruptions ou suppressions accidentelles. La restauration a pour objectif de remettre à disposition un état cohérent de la base de données après incident, en limitant l'impact opérationnel.



Étape 1 : vérification du périmètre de protection

Identifier les données couvertes par la stratégie de sauvegarde :

- Base de données métier **RDS MySQL**,
- Eventuels snapshots critiques nécessaires à la reprise,
- Jeux de données critiques nécessaires à la reprise,
- Périmètre exact des environnements concernés (**DEV, TEST-UAT, PROD**).

Résultat attendu : le périmètre des données à protéger est clairement identifié.

Étape 2 : vérification de la politique de sauvegarde active

Contrôler que la politique de sauvegarde prévue est bien en place sur l'environnement cible :

- Sauvegardes automatiques activées,
- Fenêtre de sauvegarde définie,
- Durée de rétention cohérente avec l'environnement,
- Chiffrement activé si prévu,
- Traçabilité des opérations de sauvegarde disponible.

Résultat attendu : la politique de sauvegarde active correspond aux exigences d'exploitation définies pour l'environnement concerné.

Étape 3 : vérification de l'état du service RDS avant sauvegarde

Avant toute opération sensible, contrôler :

- Que l'instance RDS est en état Available,
- Qu'aucune alerte critique n'est active,
- Qu'aucune opération de maintenance bloquante n'est en cours,
- Que les métriques principales ne signalent pas d'anomalie majeure.

Résultat attendu : **l'instance RDS** est dans un état compatible avec une sauvegarde ou un snapshot manuel.



Étape 4 : exécution ou vérification de la sauvegarde

Cas 1 : sauvegarde automatique

- Vérifier que la sauvegarde planifiée la plus récente a bien été exécutée,
- Vérifier sa date,
- Vérifier qu'elle entre bien dans la fenêtre de rétention attendue.

Cas 2 : snapshot manuel avant opération sensible

- Déclencher un snapshot manuel avant une mise à jour importante ou avant une opération à risque,
- Nommer le snapshot selon la convention définie,
- Vérifier sa création complète avant de poursuivre.

Dans **AWS**, un snapshot est une copie cohérente de l'état de la base à un instant donné, utilisée comme point de reprise.

Résultat attendu : un point de sauvegarde exploitable existe avant l'opération concernée.

Étape 5 : vérification de la complétude et la traçabilité de la sauvegarde

Après sauvegarde ou vérification d'une sauvegarde existante, contrôler :

- La présence du point de sauvegarde attendu,
- Sa date et son heure,
- L'environnement concerné,
- Sa cohérence avec la version applicative ou l'opération préparée,
- Sa traçabilité dans le journal d'exploitation.

Résultat attendu : la sauvegarde est identifiable, datée, exploitable et correctement tracée.

Étape 6 : qualification de l'incident en cas de restauration

En cas de besoin de restauration, commencer par qualifier précisément l'incident :

- Corruption de données,



- Suppression accidentelle,
- Incohérence fonctionnelle majeure,
- Erreur de migration,
- Indisponibilité partielle ou totale des données.

Il faut ensuite déterminer :

- Le périmètre de la restauration,
- Le niveau d'urgence,
- L'environnement concerné,
- Le point de restauration cible,
- L'impact sur le service et les utilisateurs.

Résultat attendu : la décision de restauration repose sur une qualification claire et documentée.

Étape 7 : identification du point de restauration cible

Choisir le point de reprise le plus pertinent :

- Sauvegarde automatique récente,
- Snapshot manuel réalisé avant changement,
- Point de restauration cohérent avec la chronologie de l'incident.

Le point retenu doit permettre de restaurer un état stable, avec une perte de données compatible avec les objectifs de reprise définis (**RPO**).

Résultat attendu : le point de restauration sélectionné est validé avant lancement.

Étape 8 : choix du mode de restauration

La restauration peut être menée selon deux approches :

- Restauration sur une instance temporaire pour validation préalable,
- Restauration directement sur l'instance cible selon la criticité et la stratégie retenue.



La restauration sur instance temporaire est généralement préférable lorsqu'une vérification préalable est souhaitée avant remise en service.

Résultat attendu : le mode de restauration est choisi et validé avant exécution.

Étape 9 : lancement de la restauration de la base

Depuis la **console AWS** ou l'outil d'exploitation retenu :

- Sélectionner le snapshot ou le point de restauration,
- Lancer la restauration vers une nouvelle instance ou l'instance prévue,
- Suivre l'avancement de l'opération,
- Attendre que l'instance restaurée soit disponible.

Selon la documentation AWS, **la restauration RDS** recrée une instance de base de données à partir du point de sauvegarde choisi. Elle n'écrase pas automatiquement l'instance d'origine sans action spécifique d'exploitation.

Résultat attendu : l'instance restaurée est créée et passe à l'état Available.

Étape 10 : vérification de l'intégrité technique de la base restaurée

Avant toute reconnexion applicative, contrôler :

- L'accessibilité de l'instance restaurée,
- La présence des schémas attendus,
- La cohérence apparente des tables critiques,
- L'absence d'erreur technique majeure,
- La cohérence du niveau de version et du schéma.

Résultat attendu : la base restaurée est techniquement exploitable.

Étape 11 : vérification de la cohérence fonctionnelle minimale

Réaliser un contrôle minimal sur les données restaurées :



- Présence de commandes récentes cohérentes,
- Présence des référentiels critiques,
- Cohérence minimale des statuts et montants,
- Absence de vide fonctionnel évident sur les objets clés.

Résultat attendu : la base restaurée semble cohérente du point de vue métier.

Étape 12 : préparation de la reconnexion contrôlée de l'application

Avant de reconnecter l'application à la base restaurée :

- Planifier la bascule,
- Informer les parties prenantes si nécessaire,
- Vérifier la configuration applicative cible,
- Confirmer que le backend pointera vers la bonne instance restaurée,
- Prévoir une fenêtre contrôlée si l'environnement de production est concerné.

Résultat attendu : la bascule applicative vers la base restaurée est préparée sans ambiguïté.

Étape 13 : reconnexion de l'application

Mettre à jour si nécessaire la configuration backend :

- Endpoint base de données,
- Identifiants associés si différents,
- Paramètres de connexion concernés.

Redéployer ou redémarrer les tâches backend si nécessaire afin de prendre en compte la nouvelle cible.

Résultat attendu : l'application est reconnectée à la base restaurée.



Étape 14 : exécution des contrôles post-restauration

Après reconnexion, vérifier :

- La stabilité des **tâches ECS**,
- Les **health checks ALB**,
- La connectivité backend vers **MySQL**,
- L'absence d'erreurs critiques dans **CloudWatch Logs**,
- Les parcours fonctionnels essentiels,
- L'absence d'anomalie immédiate visible en exploitation.

Résultat attendu : la plateforme est de nouveau opérationnelle après restauration.

Étape 15 : mesure de la reprise

Une fois l'environnement rétabli, relever :

- L'heure de détection de l'incident,
- L'heure de lancement de la restauration,
- L'heure de retour au service,
- Le volume ou périmètre de données potentiellement perdu,
- Les écarts éventuels par rapport aux objectifs de reprise.

Cela permet d'évaluer les objectifs :

- **RPO** : perte de données maximale acceptable,
- **RTO** : durée maximale acceptable de reprise.

Résultat attendu : la reprise est mesurée et objectivée.

Étape 16 : traçage de l'opération

En fin d'opération, enregistrer :

- La date et l'heure,



- L'environnement concerné,
- La cause de la restauration,
- Le point de sauvegarde utilisé,
- Le mode de restauration choisi,
- L'opérateur,
- Le résultat final,
- Les contrôles réalisés,
- Les incidents ou difficultés rencontrés.

Résultat attendu : l'opération de sauvegarde ou de restauration est historisée, traçable et auditable.

Étape 17 : capitalisation sur le retour d'expérience

Après restauration réelle ou testée, documenter :

- Les difficultés rencontrées,
- Les améliorations à apporter,
- Les délais observés,
- Les ajustements nécessaires sur la politique de sauvegarde,
- Les actions correctives ou préventives à engager.

Résultat attendu : le dispositif de reprise s'améliore au fil des retours d'expérience.

8.4.1 - Informations opérationnelles à renseigner lors de la mise en œuvre

Afin de permettre une exploitation effective de la solution une fois l'infrastructure réellement déployée, les informations opérationnelles suivantes devront être renseignées pour chaque environnement (**DEV, TEST-UAT, PROD**). Cet encadré constitue un support opérationnel à compléter lors de la phase de mise en œuvre effective de l'infrastructure et avant le passage en exploitation.



Ressources techniques de référence

- **Région AWS** utilisée,
- Nom de l'instance ou du **cluster RDS MySQL**,
- Politique de rétention configurée,
- Fenêtre de sauvegarde définie,
- Convention de nommage des snapshots manuels,
- Paramètres de connexion applicative vers la base,
- Références de supervision associées.

Chemins d'exploitation

- Accès à la **console Amazon RDS**,
- Accès aux sauvegardes automatiques,
- Accès aux snapshots manuels,
- Accès aux logs et métriques **CloudWatch**,
- Accès au **service ECS** pour reconnexion applicative après restauration.

Valeurs attendues par environnement

- Durée de rétention des sauvegardes,
- Fréquence attendue de vérification,
- Environnement autorisé pour les restaurations de test,
- Criticité métier associée,
- **Objectifs RPO / RTO** retenus.

Table de diagnostic rapide

- Absence de sauvegarde récente,
- Échec de création de snapshot,
- Instance restaurée inaccessible,
- Incohérence de schéma après restauration,
- Erreur de reconnexion applicative,
- Anomalie fonctionnelle post-restauration,
- Dépassement des objectifs de reprise.



9 - SÉCURITÉ D'EXPLOITATION

Cette section décrit les règles de sécurité opérationnelle appliquées à **HandlePizza** en exploitation.

Elle couvre la protection des secrets, le contrôle des accès aux ressources cloud et les mécanismes de chiffrement/conformité nécessaires au maintien d'un niveau de sécurité adapté en production.

9.1 - Gestion des secrets

Les secrets techniques (**identifiants DB, clés API externes, secrets JWT, tokens techniques**) sont gérés exclusivement hors code via **AWS Secrets Manager**.

Principes retenus :

- Aucun secret en clair dans le code source, les artefacts, les pipelines ou les fichiers de configuration versionnés ;
- Récupération des secrets au runtime par les services autorisés ;
- Rotation planifiée des secrets selon la politique de sécurité ;
- Journalisation et audit des accès/modifications de secrets ;
- Révocation/remplacement immédiat en cas de suspicion de compromission.

Contrôles d'exploitation :

- Vérifier régulièrement la date de dernière rotation ;
- Vérifier la portée IAM des rôles consommateurs ;
- Vérifier l'absence d'exposition de secrets dans les logs applicatifs.

9.2 - Gestion des accès IAM

Le contrôle d'accès aux **ressources AWS** s'appuie sur **IAM (Identity and Access Management)** avec application stricte du principe de moindre privilège.

Principes retenus :

- Attribution de permissions minimales par rôle technique (application, **CI/CD**, exploitation, administration) ;
- Séparation des rôles et responsabilités (pas de compte partagé privilégié non tracé) ;



- Limitation des accès humains directs aux environnements sensibles, en particulier PROD ;
- Revue périodique des droits et suppression des accès obsolètes ;
- Traçabilité des actions d'administration et d'exploitation.

Contrôles d'exploitation :

- Vérifier les **politiques IAM** critiques ;
- Contrôler les élévations de privilèges ;
- Auditer les accès à fréquence définie.

9.3 - Chiffrement et conformité

La protection des données est assurée en transit et au repos, avec des pratiques de conformité alignées sur les exigences de sécurité internes.

Chiffrement en transit :

- **HTTPS/TLS** pour les flux clients, API et intégrations externes ;
- Canaux sécurisés entre composants internes selon les capacités des services managés.

Chiffrement au repos :

- Chiffrement des données stockées (**RDS**, sauvegardes, **S3**, services de cache/messaging lorsque supporté) ;
- Protection des snapshots/sauvegardes selon la même politique.

Conformité opérationnelle :

- Masquage des données sensibles dans les logs ;
- Conservation maîtrisée des traces et journaux ;
- Application des règles de confidentialité sur les environnements non production (anonymisation/pseudonymisation des données de test).

Objectif : garantir confidentialité, intégrité et traçabilité des données sur l'ensemble du cycle d'exploitation.



10 - GLOSSAIRE

ACM (AWS Certificate Manager)	Service AWS de gestion des certificats TLS (création, renouvellement, association aux services d'exposition).
ALB (Application Load Balancer)	Equilibreur de charge applicative AWS distribuant le trafic HTTP / HTTPS vers les services backend disponibles.
AMQP (Advanced Message Queuing Protocol)	Protocole d'échange de messages utilisé par RabbitMQ/Amazon MQ.
API (Application Programming Interface)	Interface permettant à deux applications de communiquer via des contracts techniques définis.
Artefact	Résultat d'un build (image backend, bundle frontend, etc.) prêt à être déployé.
Audit trail	Historique horodaté des actions critiques (qui a fait quoi, quand, où).
Auto scaling	Mécanisme d'ajustement automatique de capacité selon la charge (CPU, mémoire, trafic, etc.)
AWS (Amazon Web Service)	Plateforme cloud utilisée pour héberger et exploiter handlePizza.
Backend	Partie serveur de l'application, porteuse des règles métier et des APIs.
Bean validation	Standard Java de validation des données entrantes (formats, contraintes, champs obligatoires).
Cache	Mécanisme de stockage temporaire en mémoire pour accélérer les lectures fréquentes.
CI/CD (Continuous integration / continuous delivery)	Chaine d'automatisation de build, tests, contrôles, déploiement et promotion des versions.
CloudFront	CDN AWS de distribution de contenus statiques avec cache en edge.
CloudWatch	Service AWS de supervision (logs, métriques, alertes, tableaux de bord).
Contrat API	Définition formelle d'une API : endpoints, méthodes, schémas, statuts,



	erreurs.
Controller	Couche backend exposant les endpoints REST et recevant les requêtes HTTP.
DTO (Data transfer object)	Objet d'échange API distinct des entités de persistance.
ECS Fargate	Service AWS d'exécution de conteneurs sans gestion direct de serveurs.
Edge web	Couche d'entrée internet recevant, filtrant, sécurisant et routant les flux vers les services internes.
ElastiCache Redis	Service AWS managé Redis, utilisé pour le cache applicatif.
Endpoint	URL d'accès à une opération API.
Environnement (DEV / TEST-UAT / PROD)	Contexte d'exécution applicatif : développement, recette, production.
Feature flag	Paramètre d'activation contrôlée d'une fonctionnalité sans redéploiement complet.
Flyway	Outil de versionnement et d'exécution des migrations de schéma de base de données.
Frontend (SPA)	Partie interface utilisateur web : ici application monopage React/TypeScript.
GitHub Actions	Service d'automatisation CI/CD intégré à GitHub.
Health check	Vérification automatique de l'état d'un service (ex. endpoint /actuator/health).
HTTPS	Protocole HTTP chiffré via TLS.
IAM (Identity and Access Management)	Service AWS de gestion des identités, rôles et permissions d'accès.
Idempotence	Propriété garantissant qu'une même opération répétée produit un résultat cohérent sans effets de bord supplémentaire.
JDBC (Java Database Connectivity)	Interface Java de connexion et d'accès aux bases relationnelles.
JPA (Java)	Spécification Java d'accès/persistance objet-relationnelles.



Persistence API)	
JWT (JSON Web Token)	Jeton signé transportant l'identité et les droits d'un utilisateur pour l'authentification API.
Latence	Temps de réponse d'un service entre requête et réponse.
Logback	Framework de journalisation Java (ici utilisé en format JSON structuré).
MapStruct	Outil de génération de mapping entre DTO et objets métier/entités.
Messaging	Echanges asynchrones entre composants via broker de messages.
Micrometer	Bibliothèque de collecte de métriques applicatives.
Monolithe modulaire	Application déployée en une unité, mais organisée en modules métier séparés.
MQ (Message Queue)	Mécanisme/file de messages pour traitements asynchrones et découplés.
MySQL 8 / RDS MySQL	Base relationnelle (service managé AWS via RDS) stockant les données métier.
Nat Gateway	Passerelle de sortie internet pour ressources privées, sans exposition entrante directe.
OpenAPI	Spécification standard de description des APIs REST.
RBAC (Role-Based Access Control)	Autorisation par rôles (client, employé, manager, direction, etc.) et périmètre.
RDS (Relationnal Database Service)	Service AWS manage pour bases relationnelles.
Redis	Base clé-valeur en mémoire utilisée pour le cache et l'accélération des accès.
Repository	Couche backend responsable des opérations d'accès aux données.
Resilience4j	Bibliothèque de résilience applicative (timeouts, retry, circuit breaker, bulkhead).
REST (Representational State Transfer)	Style d'architecture API base sur les ressources HTTP et verbes standards.
Retry	Nouvelle tentative contrôlée après échec transitoire.
Rollback	Retour à une version précédente stable après incident de



	déploiement/mise à jour.
RPO (Recovery Point Objective)	Perte de données maximale acceptable (objectif de point de reprise).
RTO (Recovery Time Objective)	Durée maximale acceptable de reprise de service.
Runbook	Procédure opérationnelle documentée de gestion d'incident/exploitation.
S3 (Simple Storage Service)	Service AWS de stockage objet, utilisé pour les assets frontend.
Secrets Manager	Service AWS de gestion sécurisée des secrets techniques (DB, JWT, API keys).
Service	Couche backend contenant la logique métier.
SLA / SLO	Objectifs d'engagement et de niveau de service (disponibilité, latence, etc.).
Spring Boot	Framework Java pour construire des applications backend et APIs rapidement.
Spring Security	Framework de sécurité backend (authentification, autorisation, filtres).
Swagger UI	Interface web générée depuis OPENAPI pour explorer et tester les endpoints.
Task Definition (ECS)	Définition technique du conteneur à exécuter (image, ressources, variables, secrets).
TLS (Transport Layer Security)	Protocole de chiffrement des communications réseau (base de HTTPS).
Timeout	Durée maximale d'attente d'une réponse avant échec contrôlé.
VPC (Virtual Private Cloud)	Réseau privé isolé dans AWS, segmenté en sous-réseaux publics/privés.
WAF (Web Application Firewall)	Pare-feu applicative web filtrant les requêtes HTTP/HTTPS malveillantes.